Visualización de voxeles con CUDA

Here is the structured Page artifact synthesizing the group chat into a usable document:

Project Plan: CUDA-Based Voxel Simulation Engine (woven_si2)

Objectives

- Develop a modular CUDA engine for voxel-based simulation and visualization.
- Support analysis of UHTC (Ultra-High Temperature Ceramics) materials and woven geometries.
- Provide a viewer similar to PicoGK but specialized for volumetric data.
- Enable simulation of thermal transfer, cooling layers, and lightweight CFD.
- Export results to CAE tools (VTK, ParaView, USD).

Base Concepts of CUDA Voxel Simulation

- Voxelization converts 3D geometric data into discrete volumetric grids (voxels) for simulation and rendering.
- CUDA enables massively parallel processing of voxel data, accelerating voxelization, raymarching, and simulation kernels.
- Efficient voxel pipelines normalize geometry, allocate 3D grids or hierarchical structures (octrees, LoDs), and map geometry to voxels using parallel threads.
- Raymarching on voxels uses CUDA kernels to traverse volumetric data for visualization and analysis.
- Thermal and physical simulations leverage CUDA for diffusion, cooling, and stress computations on voxel grids.
- Integration with OpenVDB/NanoVDB provides sparse voxel data structures optimized for GPU processing.

Deliverables

- **Core Engine**: C++20 + CUDA 12.x project with voxelization, GPU raymarching, and thermal simulation.
- **Viewer**: Real-time CUDA ↔️ OpenGL viewer with ImGui UI.

- **Voxelization Pipeline**: OpenVDB/NanoVDB integration for geometry voxelization.
- **Simulation Modules**: Thermal diffusion, cooling layers, stress analysis, CFD-lite.
- **Export Tools**: VTK, USD, NanoVDB exporters for CAE validation.
- **Validation Tools**: CPU shaders (PositionShader, NormalShader, DiffuseShader) for comparison with GPU results.

Timeline

1. **Setup (Week 1-2)**
   - Configure CMake project with dependencies (OpenVDB, NanoVDB, CUDA, GLFW, ImGui, Eigen).
   - Integrate third-party repos into third_party/.
2. **Voxelization & Data Handling (Week 3-4)**
   - Implement openvdb_loader.cpp and nanovdb_loader.cpp.
   - Add voxelization of woven geometries.
3. **Viewer Development (Week 5-6)**
   - Implement CUDA ↔️ OpenGL interop (cuda_interop.cu).
   - Build viewer with GLFW + ImGui.
   - Add raymarching kernel (raymarch.cu).
4. **Simulation Modules (Week 7-10)**
   - Implement thermal_sim.cu (GPU thermal diffusion).
   - Add cooling layers and optimizer.
   - Add stress and CFD-lite solvers.
5. **Validation & Export (Week 11-12)**
   - Implement CPU shaders with PositionShader for validation.
   - Export results to VTK and USD.
   - Compare CPU vs GPU outputs.
6. **Final Integration (Week 13-14)**
   - Assemble all modules.
   - Test with UHTC woven geometries.
   - Optimize performance and profiling.

Roles

- **Lead Developer (Víctor)**: Oversees architecture, integration, and CUDA kernels.
- **Support Developer (KiloCode)**: Implements modules per instructions, ensures modularity.
- **Validation Engineer**: Uses CPU shaders and CAE exports to validate GPU results.

Risks

- Complexity of CUDA ↔️ OpenGL interop.
- Performance bottlenecks in large voxel volumes.
- Accuracy of thermal simulation compared to CAE standards.
- Integration of multiple third-party libraries.

Repository Structure

```
woven_si2/
├── CMakeLists.txt
├── README.md
├── src/
│   ├── main.cpp
│   ├── viewer/
│   ├── cuda/
│   ├── vdb/
│   ├── geometry/
│   ├── analysis/
│   ├── export/
│   └── utils/
└── third_party/
    ├── glfw/
    ├── imgui/
    ├── openvdb/
    ├── nanovdb/
    ├── cuda_samples/
    ├── gvdb/
    └── eigen/
```

Action Items

☐ Configure CMake project with dependencies.

☐ Copy essential files from OpenVDB, NanoVDB, CUDA Samples, VolumeRender_CUDA, GVDB, GLFW, ImGui, Taichi, VTK.

☐ Implement voxel loaders and converters.

☐ Implement CUDA raymarching and interop.

☐ Build viewer with ImGui UI.

☐ Implement thermal simulation and cooling layers.

☐ Add CPU shaders for validation.

☐ Implement export modules.

☐ Validate with test geometries and volumes.

Appendix: Essential Files by Repository

OpenVDB

- tools/VolumeToMesh.h, LevelSetUtil.h, GridTransformer.h, Interpolation.h, Filter.h, Prune.h
- tools/VolumeRendering.h, RayTracer.h, RayIntersector.h, PositionShader.h, NormalShader.h, DiffuseShader.h, BaseShader.h
- tools/BaseCamera.h, PerspectiveCamera.h, OrthographicCamera.h, Film.h
- math/*, io/*

NanoVDB

- NanoVDB.h, CudaGrid.h, CudaDeviceBuffer.h, util/*
- examples/ex_render_nanovdb_cuda/*, examples/ex_read_nanovdb_sphere_accessor_cuda/*

CUDA Samples

- simpleGL.cu, rendercheck_gl.cpp, gl_helper.h, main.cpp

VolumeRender_CUDA

- volume.cu, gl_interop.cpp, main.cpp, shader/display.frag

GVDB Voxels

- gvdb_core.cpp, gvdb_sim.cpp, gvdb_volume.cpp, cuda/gvdb_cuda.cu, cuda/gvdb_raytrace.cu

GLFW

- glfw3.h, context.c, window.c, input.c

ImGui

- imgui.cpp, imgui_draw.cpp, imgui_widgets.cpp, backends/imgui_impl_glfw.cpp, backends/imgui_impl_opengl3.cpp

Taichi

- physics/*, backends/cuda/*, algorithms/*

VTK

- IO/XML/*, Common/DataModel/*, Filters/Core/*

Future Considerations

- Advanced viewer features: slicing, iso-surfaces, colormaps.
- Integration with Omniverse via USD export.
- Performance profiling and optimization for large-scale simulations.
- Extension to multi-GPU setups.

This document provides KiloCode and the team with a clear roadmap, repository structure, essential files, and actionable instructions to build the CUDA voxel simulation engine woven_si2.

Project Plan: CUDA-Based Voxel Simulation Engine (woven_si2)

Objectives

- Develop a modular CUDA engine for voxel-based simulation and visualization.
- Support analysis of UHTC (Ultra-High Temperature Ceramics) materials and woven geometries.
- Provide a viewer similar to PicoGK but specialized for volumetric data.
- Enable simulation of thermal transfer, cooling layers, and lightweight CFD.
- Export results to CAE tools (VTK, ParaView, USD).

Deliverables

- **Core Engine**: C++20 + CUDA 12.x project with voxelization, GPU raymarching, and thermal simulation.
- **Viewer**: Real-time CUDA ↔️ OpenGL viewer with ImGui UI.
- **Voxelization Pipeline**: OpenVDB/NanoVDB integration for geometry voxelization.
- **Simulation Modules**: Thermal diffusion, cooling layers, stress analysis, CFD-lite.
- **Export Tools**: VTK, USD, NanoVDB exporters for CAE validation.

- **Validation Tools**: CPU shaders (PositionShader, NormalShader, DiffuseShader) for comparison with GPU results.

Timeline

1. **Setup (Week 1-2)**
   - Configure CMake project with dependencies (OpenVDB, NanoVDB, CUDA, GLFW, ImGui, Eigen).
   - Integrate third-party repos into third_party/.
2. **Voxelization & Data Handling (Week 3-4)**
   - Implement openvdb_loader.cpp and nanovdb_loader.cpp.
   - Add voxelization of woven geometries.
3. **Viewer Development (Week 5-6)**
   - Implement CUDA ↔️ OpenGL interop (cuda_interop.cu).
   - Build viewer with GLFW + ImGui.
   - Add raymarching kernel (raymarch.cu).
4. **Simulation Modules (Week 7-10)**
   - Implement thermal_sim.cu (GPU thermal diffusion).
   - Add cooling layers and optimizer.
   - Add stress and CFD-lite solvers.
5. **Validation & Export (Week 11-12)**
   - Implement CPU shaders with PositionShader for validation.
   - Export results to VTK and USD.
   - Compare CPU vs GPU outputs.
6. **Final Integration (Week 13-14)**
   - Assemble all modules.
   - Test with UHTC woven geometries.
   - Optimize performance and profiling.

Roles

- **Lead Developer (Víctor)**: Oversees architecture, integration, and CUDA kernels.
- **Support Developer (KiloCode)**: Implements modules per instructions, ensures modularity.
- **Validation Engineer**: Uses CPU shaders and CAE exports to validate GPU results.

Risks

- Complexity of CUDA ↔️ OpenGL interop.
- Performance bottlenecks in large voxel volumes.
- Accuracy of thermal simulation compared to CAE standards.
- Integration of multiple third-party libraries.

Repository Structure

```
woven_si2/
├── CMakeLists.txt
├── README.md
├── src/
│   ├── main.cpp
│   ├── viewer/
│   ├── cuda/
│   ├── vdb/
│   ├── geometry/
│   ├── analysis/
│   ├── export/
│   └── utils/
└── third_party/
    ├── glfw/
    ├── imgui/
    ├── openvdb/
    ├── nanovdb/
    ├── cuda_samples/
    ├── gvdb/
    └── eigen/
```

Action Items

☐ Configure CMake project with dependencies.

☐ Copy essential files from OpenVDB, NanoVDB, CUDA Samples, VolumeRender_CUDA, GVDB, GLFW, ImGui, Taichi, VTK.

☐ Implement voxel loaders and converters.

☐ Implement CUDA raymarching and interop.

☐ Build viewer with ImGui UI.

☐ Implement thermal simulation and cooling layers.

☐ Add CPU shaders for validation.

☐ Implement export modules.

☐ Validate with test geometries and volumes.

Appendix: Essential Files by Repository

OpenVDB

- tools/VolumeToMesh.h, LevelSetUtil.h, GridTransformer.h, Interpolation.h, Filter.h, Prune.h
- tools/VolumeRendering.h, RayTracer.h, RayIntersector.h, PositionShader.h, NormalShader.h, DiffuseShader.h, BaseShader.h
- tools/BaseCamera.h, PerspectiveCamera.h, OrthographicCamera.h, Film.h
- math/*, io/*

NanoVDB

- NanoVDB.h, CudaGrid.h, CudaDeviceBuffer.h, util/*
- examples/ex_render_nanovdb_cuda/*, examples/ex_read_nanovdb_sphere_accessor_cuda/*

CUDA Samples

- simpleGL.cu, rendercheck_gl.cpp, gl_helper.h, main.cpp

VolumeRender_CUDA

- volume.cu, gl_interop.cpp, main.cpp, shader/display.frag

GVDB Voxels

- gvdb_core.cpp, gvdb_sim.cpp, gvdb_volume.cpp, cuda/gvdb_cuda.cu, cuda/gvdb_raytrace.cu

GLFW

- glfw3.h, context.c, window.c, input.c

ImGui

- imgui.cpp, imgui_draw.cpp, imgui_widgets.cpp, backends/imgui_impl_glfw.cpp, backends/imgui_impl_opengl3.cpp

Taichi

- physics/*, backends/cuda/*, algorithms/*

VTK

- IO/XML/*, Common/DataModel/*, Filters/Core/*

Future Considerations

- Advanced viewer features: slicing, iso-surfaces, colormaps.
- Integration with Omniverse via USD export.

- Performance profiling and optimization for large-scale simulations.
- Extension to multi-GPU setups.

This document provides KiloCode and the team with a clear roadmap, repository structure, essential files, and actionable instructions to build the CUDA voxel simulation engine woven_si2.

Relevant CUDA Repositories for Geometric Simulation and Aerospace Applications

To complement the woven_si2 project, here are several relevant open-source CUDA repositories and resources that provide valuable codebases, examples, and methodologies for voxelization, geometric simulation, and aerospace-related computations:

1. GPU-VPM (Vortex Particle Method) - dominikkau/CIS5650-Final-Project-GPU-VPM

- GitHub: https://github.com/dominikkau/CIS5650-Final-Project-GPU-VPM
- Description: A CUDA-based GPU implementation of the Vortex Particle Method for aerodynamic simulations, suitable for early-stage aerospace vehicle design.
- Features: Parallelized vortex particle calculations, Runge-Kutta integration, modular CUDA kernels.

2. CUDA Mesh Voxelization - bigmat18/cuda-mesh-voxelization

- GitHub: https://github.com/bigmat18/cuda-mesh-voxelization
- Description: GPU-accelerated pipeline for robust 3D mesh voxelization and Boolean operations using Signed Distance Fields (SDFs).
- Features: Mesh voxelization, CSG operations, SDF computation, benchmarking.

3. CUDA Voxel Mapping - jorgenfj/cuda-voxel-mapping

- GitHub: https://github.com/jorgenfj/cuda-voxel-mapping
- Description: CUDA-accelerated library for 3D environment voxel mapping, distance field calculations, and real-time sensor data integration.
- Features: Sparse voxel chunk mapping, Euclidean Distance Transform, raycasting kernel.

4. DEM-Engine - projectchrono/DEM-Engine

- GitHub: https://github.com/projectchrono/DEM-Engine
- Description: Dual-GPU Discrete Element Method solver with complex grain geometry support, useful for granular material simulations.

- Features: Multi-GPU support, customizable contact force models, deformable meshes.

5. NVIDIA GVDB Voxels - NVIDIA/gvdb-voxels

- GitHub: https://github.com/NVIDIA/gvdb-voxels
- Description: NVIDIA's GVDB Voxels library for sparse volume compute and rendering on GPUs, optimized for volumetric data.
- Features: Sparse voxel octrees, GPU raytracing, volume rendering, multi-context support.

These repositories provide practical CUDA implementations and examples that can be leveraged or referenced to enhance the woven_si2 engine, especially for voxelization, volumetric rendering, and simulation tasks relevant to aerospace materials and geometries.

Project Plan: CUDA-Based Voxel Simulation Engine (woven_si2)

Objectives

- Develop a modular CUDA engine for voxel-based simulation and visualization.
- Support analysis of UHTC (Ultra-High Temperature Ceramics) materials and woven geometries.
- Provide a viewer similar to PicoGK but specialized for volumetric data.
- Enable simulation of thermal transfer, cooling layers, and lightweight CFD.
- Export results to CAE tools (VTK, ParaView, USD).

Base Concepts of CUDA Voxel Simulation

- Voxelization converts 3D geometric data into discrete volumetric grids (voxels) for simulation and rendering.
- CUDA enables massively parallel processing of voxel data, accelerating voxelization, raymarching, and simulation kernels.
- Efficient voxel pipelines normalize geometry, allocate 3D grids or hierarchical structures (octrees, LoDs), and map geometry to voxels using parallel threads.
- Raymarching on voxels uses CUDA kernels to traverse volumetric data for visualization and analysis.
- Thermal and physical simulations leverage CUDA for diffusion, cooling, and stress computations on voxel grids.

- Integration with OpenVDB/NanoVDB provides sparse voxel data structures optimized for GPU processing.

Deliverables

- **Core Engine**: C++20 + CUDA 12.x project with voxelization, GPU raymarching, and thermal simulation.
- **Viewer**: Real-time CUDA ↔️ OpenGL viewer with ImGui UI.
- **Voxelization Pipeline**: OpenVDB/NanoVDB integration for geometry voxelization.
- **Simulation Modules**: Thermal diffusion, cooling layers, stress analysis, CFD-lite.
- **Export Tools**: VTK, USD, NanoVDB exporters for CAE validation.
- **Validation Tools**: CPU shaders (PositionShader, NormalShader, DiffuseShader) for comparison with GPU results.

Timeline

1. **Setup (Week 1-2)**
   - Configure CMake project with dependencies (OpenVDB, NanoVDB, CUDA, GLFW, ImGui, Eigen).
   - Integrate third-party repos into third_party/.
2. **Voxelization & Data Handling (Week 3-4)**
   - Implement openvdb_loader.cpp and nanovdb_loader.cpp.
   - Add voxelization of woven geometries.
3. **Viewer Development (Week 5-6)**
   - Implement CUDA ↔️ OpenGL interop (cuda_interop.cu).
   - Build viewer with GLFW + ImGui.
   - Add raymarching kernel (raymarch.cu).
4. **Simulation Modules (Week 7-10)**
   - Implement thermal_sim.cu (GPU thermal diffusion).
   - Add cooling layers and optimizer.
   - Add stress and CFD-lite solvers.
5. **Validation & Export (Week 11-12)**
   - Implement CPU shaders with PositionShader for validation.
   - Export results to VTK and USD.
   - Compare CPU vs GPU outputs.

6. **Final Integration (Week 13-14)**
    - Assemble all modules.
    - Test with UHTC woven geometries.
    - Optimize performance and profiling.

Roles

- **Lead Developer (Víctor)**: Oversees architecture, integration, and CUDA kernels.
- **Support Developer (KiloCode)**: Implements modules per instructions, ensures modularity.
- **Validation Engineer**: Uses CPU shaders and CAE exports to validate GPU results.

Risks

- Complexity of CUDA ↔️ OpenGL interop.
- Performance bottlenecks in large voxel volumes.
- Accuracy of thermal simulation compared to CAE standards.
- Integration of multiple third-party libraries.

Repository Structure

```
woven_si2/
├── CMakeLists.txt
├── README.md
├── src/
│   ├── main.cpp
│   ├── viewer/
│   ├── cuda/
│   ├── vdb/
│   ├── geometry/
│   ├── analysis/
│   ├── export/
│   └── utils/
└── third_party/
    ├── glfw/
    ├── imgui/
    ├── openvdb/
    ├── nanovdb/
    ├── cuda_samples/
    ├── gvdb/
    └── eigen/
```

Action Items

☐ Configure CMake project with dependencies.

☐ Copy essential files from OpenVDB, NanoVDB, CUDA Samples, VolumeRender_CUDA, GVDB, GLFW, ImGui, Taichi, VTK.

☐ Implement voxel loaders and converters.

☐ Implement CUDA raymarching and interop.

☐ Build viewer with ImGui UI.

☐ Implement thermal simulation and cooling layers.

☐ Add CPU shaders for validation.

☐ Implement export modules.

☐ Validate with test geometries and volumes.

Appendix: Essential Files by Repository

OpenVDB

- tools/VolumeToMesh.h, LevelSetUtil.h, GridTransformer.h, Interpolation.h, Filter.h, Prune.h
- tools/VolumeRendering.h, RayTracer.h, RayIntersector.h, PositionShader.h, NormalShader.h, DiffuseShader.h, BaseShader.h
- tools/BaseCamera.h, PerspectiveCamera.h, OrthographicCamera.h, Film.h
- math/*, io/*

NanoVDB

- NanoVDB.h, CudaGrid.h, CudaDeviceBuffer.h, util/*
- examples/ex_render_nanovdb_cuda/*, examples/ex_read_nanovdb_sphere_accessor_cuda/*

CUDA Samples

- simpleGL.cu, rendercheck_gl.cpp, gl_helper.h, main.cpp

VolumeRender_CUDA

- volume.cu, gl_interop.cpp, main.cpp, shader/display.frag

GVDB Voxels

- gvdb_core.cpp, gvdb_sim.cpp, gvdb_volume.cpp, cuda/gvdb_cuda.cu, cuda/gvdb_raytrace.cu

GLFW

- glfw3.h, context.c, window.c, input.c

ImGui

- imgui.cpp, imgui_draw.cpp, imgui_widgets.cpp, backends/imgui_impl_glfw.cpp, backends/imgui_impl_opengl3.cpp

Taichi

- physics/*, backends/cuda/*, algorithms/*

VTK

- IO/XML/*, Common/DataModel/*, Filters/Core/*

Future Considerations

- Advanced viewer features: slicing, iso-surfaces, colormaps.
- Integration with Omniverse via USD export.
- Performance profiling and optimization for large-scale simulations.
- Extension to multi-GPU setups.

This document provides KiloCode and the team with a clear roadmap, repository structure, essential files, and actionable instructions to build the CUDA voxel simulation engine woven_si2.

Project Plan: CUDA-Based Voxel Simulation Engine (woven_si2)

Objectives

- Develop a modular CUDA engine for voxel-based simulation and visualization.
- Support analysis of UHTC (Ultra-High Temperature Ceramics) materials and woven geometries.
- Provide a viewer similar to PicoGK but specialized for volumetric data.
- Enable simulation of thermal transfer, cooling layers, and lightweight CFD.
- Export results to CAE tools (VTK, ParaView, USD).

Deliverables

- **Core Engine**: C++20 + CUDA 12.x project with voxelization, GPU raymarching, and thermal simulation.
- **Viewer**: Real-time CUDA ↔️ OpenGL viewer with ImGui UI.
- **Voxelization Pipeline**: OpenVDB/NanoVDB integration for geometry voxelization.
- **Simulation Modules**: Thermal diffusion, cooling layers, stress analysis, CFD-lite.
- **Export Tools**: VTK, USD, NanoVDB exporters for CAE validation.
- **Validation Tools**: CPU shaders (PositionShader, NormalShader, DiffuseShader) for comparison with GPU results.

Timeline

1. **Setup (Week 1-2)**
   - Configure CMake project with dependencies (OpenVDB, NanoVDB, CUDA, GLFW, ImGui, Eigen).
   - Integrate third-party repos into third_party/.
2. **Voxelization & Data Handling (Week 3-4)**
   - Implement openvdb_loader.cpp and nanovdb_loader.cpp.
   - Add voxelization of woven geometries.
3. **Viewer Development (Week 5-6)**
   - Implement CUDA ↔️ OpenGL interop (cuda_interop.cu).
   - Build viewer with GLFW + ImGui.

- Add raymarching kernel (raymarch.cu).

4. **Simulation Modules (Week 7-10)**
   - Implement thermal_sim.cu (GPU thermal diffusion).
   - Add cooling layers and optimizer.
   - Add stress and CFD-lite solvers.
5. **Validation & Export (Week 11-12)**
   - Implement CPU shaders with PositionShader for validation.
   - Export results to VTK and USD.
   - Compare CPU vs GPU outputs.
6. **Final Integration (Week 13-14)**
   - Assemble all modules.
   - Test with UHTC woven geometries.
   - Optimize performance and profiling.

Roles

- **Lead Developer (Víctor)**: Oversees architecture, integration, and CUDA kernels.
- **Support Developer (KiloCode)**: Implements modules per instructions, ensures modularity.
- **Validation Engineer**: Uses CPU shaders and CAE exports to validate GPU results.

Risks

- Complexity of CUDA ↔️ OpenGL interop.
- Performance bottlenecks in large voxel volumes.
- Accuracy of thermal simulation compared to CAE standards.
- Integration of multiple third-party libraries.

Repository Structure

```
woven_si2/
 ├── CMakeLists.txt
 ├── README.md
 ├── src/
 │   ├── main.cpp
 │   ├── viewer/
 │   ├── cuda/
 │   ├── vdb/
 │   ├── geometry/
 │   ├── analysis/
 │   ├── export/
 │   └── utils/
 └── third_party/
     ├── glfw/
     ├── imgui/
     ├── openvdb/
     ├── nanovdb/
     ├── cuda_samples/
     ├── gvdb/
     └── eigen/
```

Action Items

☐ Configure CMake project with dependencies.

☐ Copy essential files from OpenVDB, NanoVDB, CUDA Samples, VolumeRender_CUDA, GVDB, GLFW, ImGui, Taichi, VTK.

☐ Implement voxel loaders and converters.

☐ Implement CUDA raymarching and interop.

☐ Build viewer with ImGui UI.

☐ Implement thermal simulation and cooling layers.

☐ Add CPU shaders for validation.

☐ Implement export modules.

☐ Validate with test geometries and volumes.

Appendix: Essential Files by Repository

OpenVDB

- tools/VolumeToMesh.h, LevelSetUtil.h, GridTransformer.h, Interpolation.h, Filter.h, Prune.h
- tools/VolumeRendering.h, RayTracer.h, RayIntersector.h, PositionShader.h, NormalShader.h, DiffuseShader.h, BaseShader.h
- tools/BaseCamera.h, PerspectiveCamera.h, OrthographicCamera.h, Film.h
- math/*, io/*

NanoVDB

- NanoVDB.h, CudaGrid.h, CudaDeviceBuffer.h, util/*
- examples/ex_render_nanovdb_cuda/*, examples/ex_read_nanovdb_sphere_accessor_cuda/*

CUDA Samples

- simpleGL.cu, rendercheck_gl.cpp, gl_helper.h, main.cpp

VolumeRender_CUDA

- volume.cu, gl_interop.cpp, main.cpp, shader/display.frag

GVDB Voxels

- gvdb_core.cpp, gvdb_sim.cpp, gvdb_volume.cpp, cuda/gvdb_cuda.cu, cuda/gvdb_raytrace.cu

GLFW

- glfw3.h, context.c, window.c, input.c

ImGui

- imgui.cpp, imgui_draw.cpp, imgui_widgets.cpp, backends/imgui_impl_glfw.cpp, backends/imgui_impl_opengl3.cpp

Taichi

- physics/*, backends/cuda/*, algorithms/*

VTK

- IO/XML/*, Common/DataModel/*, Filters/Core/*

Future Considerations

- Advanced viewer features: slicing, iso-surfaces, colormaps.
- Integration with Omniverse via USD export.
- Performance profiling and optimization for large-scale simulations.
- Extension to multi-GPU setups.

This document provides KiloCode and the team with a clear roadmap, repository structure, essential files, and actionable instructions to build the CUDA voxel simulation engine woven_si2.